TECHNICAL METHODS REPORT

# Methods, Code and Reproducibility Guide

## Kenya Electricity Demand Outlook and Supply-Security Assessment

*Every step taken to produce the final report: data lineage, models and formulas, parameters, assumptions, validation, problems met and how they were resolved, and the full source code.*

Author: Samatar Ahmed

Date: October 2026

Companion document: Final Report (25 pages). Repository: see README.

# 1 Purpose and how to use this document

The Final Report presents findings. This document records how they were produced so that a reviewer can audit, rerun and extend the work. It is organised in the order of the pipeline. Each stage states its inputs, the transformation and the exact formulas, the parameters chosen, the checks applied and the outputs written. Section 10 is a decision-and-issue log, including mistakes caught during the work; Appendices E–H contain the full code.

**Authorship and tooling note**

The pipeline was developed with an AI coding assistant (Claude) running the code and drafting text, with data supplied by the author. The author should be able to explain every method in this document; the formulas and decision log are written to make that possible.

## 1.1 Research questions

- RQ1 How fast will Kenya's electricity demand grow to 2035, and how uncertain is that?
- RQ2 How secure is Kenya's electricity supply structurally, relative to regional peers, and what has changed?
- RQ3 What happens to annual energy balance if hydrology fails or imports are lost?
- RQ4 How much new generation is needed by 2030 and 2035 under alternative demand paths and technology mixes?

# 2 Environment and tooling

**Table 1. Software environment**

| Component | Version |
|---|---|
| Python | 3.13.16 |
| pandas | 3.0.5 |
| numpy | 2.5.3 |
| scipy | 1.18.1 |
| matplotlib | 3.11.2 |
| python-docx | 1.2.0 |
| LibreOffice (DOCX→PDF) | 24.2 |
| OS | Linux (Linux container) |

statsmodels could not be installed because the workspace had no route to PyPI. All time-series models (random walk with drift, log-linear trend, ARIMA(1,1,0) with drift, damped-trend ETS) and the clustered-standard-error regression were therefore implemented directly with numpy and scipy. This is a transparency gain: every line of each estimator is in Appendix F. A reviewer wishing to cross-check could run the same specifications in statsmodels; small differences in ETS parameter estimates are expected, because the smoothing parameters here are chosen by minimising squared one-step errors with bounded L-BFGS-B from three starting points.

## 2.1 Reproducibility conventions

- Scripts are run from the repository root with python3 -I (isolated mode), so no stray module in the working directory or environment can alter behaviour.
- Random draws use numpy's default_rng with a fixed seed (20261012).
- Each stage reads only the previous stage's files; no stage depends on global state.
- Raw data are never modified; processed data are regenerated by Stage 1.

# 3 Repository layout and pipeline overview

```
kenya-electricity-outlook/
  README.md
  requirements.txt
  run_all.sh
  data/raw/         ember_generation_yearly.csv, owid-energy-data.csv   (unmodified
downloads)
  data/processed/   kenya_annual.csv, regional_panel.csv
  src/              01_build_dataset.py ... 06_build_technical_report.py, docx_helpers.py
  outputs/tables/   backtest_*.csv, demand_forecast_*.csv, security_index_panel.csv,
                    stress_scenarios_2025.csv, capacity_gap_scenarios.csv, *.json
  outputs/figures/  fig01 ... fig11 (PNG, 200 dpi)
  reports/          Final report and this report (DOCX and PDF)
  docs/             data dictionary, assumptions, CV bullets
```

**Table 2. Pipeline stages**

| Stage | Script | Inputs | Outputs |
|---|---|---|---|
| 1 | 01_build_dataset.py | Raw Ember + OWID CSVs | regional_panel.csv, kenya_annual.csv, data_quality.json, data_reconciliation_kenya.csv |
| 2 | 02_forecast_models.py | kenya_annual.csv, regional_panel.csv | backtest_*.csv, elasticities.json, demand_forecast_*.csv, forecast_facts.json |
| 3 | 03_security_stress_scenarios.py | regional_panel.csv, demand_forecast_*.csv | security_index_panel.csv, stress_scenarios_2025.csv, capacity_gap_scenarios.csv, regional_benchmark_2024.csv, stage3_facts.json |
| 4 | 04_figures.py | Processed data and all tables | outputs/figures/*.png |
| 5 | 05/06_build_*_report.py | Tables, figures, JSON facts | reports/*.docx |
| 6 | 07_build_dashboard.py | Tables + src/dashboard_template.html | index.html |

All numbers quoted in the Final Report are inserted programmatically from the JSON and CSV outputs by the report builders, so text and tables cannot drift from the analysis. Where the text was written by hand, claims were checked against the data (Section 10).

# 4 Stage 1: data acquisition and harmonisation

## 4.1 Inputs

Table 3. Raw datasets

| File | Provider | Shape | Licence |
|---|---|---|---|
| ember_generation_yearly.csv | Ember | 104,273 rows × 24 cols; 1985–2025; long format with one row per area, year and electricity source | CC BY 4.0 |
| owid-energy-data.csv | Our World in Data | 23,377 rows × 130 cols; 1900–2025; wide format, one row per country-year | CC BY 4.0 |

## 4.2 Steps

- Filter Ember to Area type = 'Country or economy' and ten countries: Kenya, Uganda, Tanzania (Ember name 'Tanzania (the United Republic of)'), Ethiopia, Rwanda, Burundi, Zambia, Mozambique, Congo (DRC) and Sudan. South Sudan and Somalia were excluded because of short or sparse series.
- Pivot generation to one row per country-year with sources as columns; fill missing coal, gas and nuclear with zero (they are structural zeros, not missing data).
- Rename 'Other renewables' to geothermal_other_ren, and 'Total generation' to generation; pull power-sector emissions and intensity from the 'Total generation' rows.
- Merge population, GDP and energy per capita from OWID by country-year (DR Congo name aligned).
- Derive demand_per_capita_kwh = demand × 10^9 / population and gdp_per_capita = gdp / population.
- Write regional_panel.csv (252 rows × 26 columns) and kenya_annual.csv (26 rows).

## 4.3 Checks

Three accounting identities and one cross-source comparison were evaluated for Kenya for all years; results are in the Final Report, Table 2. All identities held to within 0.01%, and Ember and OWID matched within 0.06%, which is expected because OWID derives its electricity series from Ember.

## 4.4 Data issues identified

- Kenya's geothermal capacity is reported as zero in Ember although generation is 6.4 TWh (2025). Capacity-based reserve margins were therefore not computed; capacity is used only for thermal plant.
- OWID GDP for Kenya ends in 2022.
- Direct download from GitHub raw, the World Bank API and the Ember website was blocked by the workspace's network policy; the author supplied the files manually. No data were simulated.

# 5 Stage 2: demand forecasting

## 5.1 Target and transformation

The target is electricity demand D_t (TWh), 2000–2025, modelled as y_t = ln D_t, so that all models describe proportional growth and forecasts are positive by construction.

## 5.2 Models

**Table 4. Forecasting models (all on log demand)**

| Model | Specification | Parameters |
|---|---|---|
| Drift (RW+drift) | ŷ_{T+h} = y_T + h · mean(Δy) | None |
| Recent drift | ŷ_{T+h} = y_T + h · mean of last 8 Δy | Window w = 8 |
| Log-linear trend | ŷ_{T+h} = a + b(T+h), OLS on t = 0..T | a, b |
| ARIMA(1,1,0)+drift | Δy_t = c + φ Δy_{t-1} + ε_t, estimated by conditional least squares; φ clipped to ±0.95; forecasts iterate the recursion | c, φ |
| ETS(A,Ad,N) on logs | ℓ_t = ℓ_{t-1}+φb_{t-1}+αe_t; b_t = φb_{t-1}+αβe_t; ŷ_{T+h} = ℓ_T + (φ+…+φ^h) b_T | α∈[.01,.99], β∈[.01,.99], φ∈[.80,.98] by SSE minimisation |
| GDP-driver (conditional) | Δy_t = a + b Δln GDP_t; forecast uses realised GDP in backtest | a, b |
| Ensemble (mean) | Mean of log-forecasts of Drift, ARIMA and Log-linear trend | Equal weights |

## 5.3 Backtest design

Rolling-origin evaluation: for each origin year T from 2010 to 2024 (minimum training window 11 observations, expanding thereafter), each model was refitted using data up to T only and forecasts for h = 1, 2, 3 were recorded against the realised value. This yields 42 out-of-sample forecasts per univariate model (15, 14 and 13 at horizons 1, 2 and 3) and 33 for the GDP-driver model, whose targets are limited to years with GDP (≤ 2022). Metrics: MAPE, RMSE (TWh), mean percentage bias, sd of log errors and MAE relative to the drift model.

## 5.4 Prediction intervals

Let s_1 be the RMS of the ensemble's one-step log errors from the backtest (0.0410). The h-step log-forecast standard deviation is assumed to be s_1·√h, and intervals are exp(ln F_h ± z·s_1√h) with z = 1.2816 (80%) and 1.96 (95%). Empirical coverage in the backtest: 80% intervals covered 87%, 93% and 100% of outcomes at h = 1, 2, 3 (n = 15, 14, 13). Beyond h = 3 the √h scaling is an untested assumption.

## 5.5 Ensemble selection and its bias

The first run used a four-member ensemble including ETS and recent-drift. Backtests showed those two were weakest at 2–3 year horizons (ETS 9.0% MAPE at h = 3), so the ensemble was changed to drift, ARIMA and log-linear trend. This is a selection step on the same data used to evaluate the result, so reported ensemble accuracy is optimistic. The forecast level is robust to the choice: retained members span 24.0–25.6 TWh in 2035.

## 5.6 Elasticity estimation

- Growth form: $\Delta \ln D_{it} = \alpha + \beta \Delta \ln GDP_{it} + u$. Pooled over nine countries with country-clustered standard errors (CR1 small-sample correction $G/(G-1)\cdot(n-1)/(n-k)$): $\beta = 0.346$, SE = 0.186, n = 198.
- Kenya only, classical OLS: $\beta = 0.561$, SE = 0.599, p = 0.36, $R^2 = 0.042$, n = 22.
- Levels form with country fixed effects (within transformation): $\ln(D/pop)_{it} = \alpha_i + \beta \ln(GDP/pop)_{it}$: $\beta = 0.845$, n = 207. Standard errors not reported for this descriptive estimate.

With nine clusters, cluster-robust inference is itself unreliable; the estimates are presented as descriptive.

## 5.7 Backtest results

**Table 5. Backtest metrics (MAPE and bias in %, RMSE in TWh)**

| Model | MAPE h1 | RMSE h1 | Bias% h1 | MAPE h2 | RMSE h2 | Bias% h2 | MAPE h3 | RMSE h3 | Bias% h3 |
|---|---|---|---|---|---|---|---|---|---|
| Ensemble (mean) | 2.90 | 0.41 | -1.0 | 3.93 | 0.48 | -1.2 | 3.34 | 0.42 | -1.5 |
| Drift (RW+drift) | 3.39 | 0.44 | -0.1 | 4.31 | 0.52 | -0.1 | 3.03 | 0.45 | +0.1 |
| Log-linear trend | 3.17 | 0.43 | -2.3 | 3.59 | 0.48 | -2.9 | 3.95 | 0.54 | -3.5 |
| GDP-driver (conditional) | 3.40 | 0.46 | -0.6 | 4.58 | 0.54 | -1.1 | 4.12 | 0.51 | -1.7 |
| ARIMA(1,1,0)+drift | 3.58 | 0.47 | -0.4 | 4.82 | 0.59 | -0.7 | 3.50 | 0.51 | -0.9 |
| Recent drift (8y) | 3.70 | 0.47 | -0.5 | 4.94 | 0.63 | -0.9 | 5.03 | 0.73 | -1.1 |
| ETS damped trend | 4.42 | 0.57 | -3.3 | 6.33 | 0.84 | -6.1 | 9.04 | 1.12 | -8.9 |

# 6 Stage 3: supply-security index, stress tests and capacity gap

## 6.1 Index construction

For each country-year (2006 onward, so the volatility window is available) compute four raw indicators:

- Import dependence  I = max(0, net imports / demand) × 100.
- Concentration  HHI = Σ_j $s_j^2$, over nine source shares s_j of generation (hydro, geothermal/other renewables, wind, solar, bioenergy, coal, gas, other fossil, nuclear).
- Fuel exposure  F = fossil generation / total generation × 100.
- Hydrology risk  H = hydro share (%) × σ_8, where σ_8 is the rolling standard deviation of annual hydro growth over the last eight years (minimum five). Units: percentage of generation exposed to a one-σ hydrology swing.

Each indicator x is winsorised at the pooled 5th and 95th percentiles, then scored s = 100 × (1 − (x − lo)/(hi − lo)), so higher is more secure. The index is the arithmetic mean of the four scores (SSI) and, as a robustness check, the geometric mean (SSI_geo). Pooled scaling means scores are comparable across countries and years.

## 6.2 Robustness to weights

10,000 weight vectors were drawn from Dirichlet(1,1,1,1) and Kenya's 2024 rank recomputed for each: median 1, first in 54% of draws, 5th–95th percentile 1–7. Corner checks (70% on one pillar, 10% on others) gave Kenya ranks 7 (imports), 1 (concentration), 3 (fuel exposure), 1 (hydrology). The corner checks were run interactively while verifying the report text and are reported in the Final Report, Section 5.4.

## 6.3 Stress tests

Base: Kenya's 2025 supply stack. Thermal ceiling = 0.57 GW × 8.76 TWh/GW × CF_th, with CF_th = 0.65 → 3.246 TWh; 2025 thermal output 1.398 TWh; headroom 1.848 TWh. For a scenario with hydro shock a_h, wind shock a_w, geothermal shock a_g and import multiplier m: lost = −a_h·Hydro − a_w·Wind − a_g·Geo + (1 − m)·Imports; thermal_add = min(lost, headroom); unserved = lost − thermal_add; extra CO2 = thermal_add × 0.658 Mt/TWh (Ember 'Other fossil' intensity, 658 g/kWh).

**Table 6. Deterministic scenarios**

| ID | Shocks | Rationale |
|---|---|---|
| S1 | Hydro −25% | Moderate drought; about the 2009-sized shock (−29% vs trend) |
| S2 | Hydro −42% | Equals 2000 deviation from 5-year centred mean |
| S3 | Imports × 0 | Total interconnector loss |
| S4 | Hydro −25%, imports × 0.5 | Moderate compound |
| S5 | Hydro −42%, imports × 0 | Severe compound |
| S6 | S5 + wind −20% + geothermal −10% | Extreme compound; illustrative |

Monte Carlo: for each of 20,000 draws, sample one historical year and take hydro and geothermal deviations from their centred five-year means (joint draw, preserving co-movement); apply them to the 2025 hydro and geothermal output; compute lost energy (negative changes only), add the full import volume in the 'imports interrupted' state, and subtract thermal headroom to obtain unserved energy. Probabilities are conditional on the import state; no probability is assigned to an import outage.

## 6.4 Capacity-gap scenarios

Demand paths: forecast lower 80% bound, ensemble, upper 80% bound. Gap_t = D_t − D_2025 (imports and thermal held at 2025 volumes). New capacity for technology j with energy share w_j and capacity factor CF_j: GW = Σ_j Gap·w_j / (8.76·CF_j). CF: geothermal 0.85, wind 0.45, solar 0.25. Mixes (geothermal/wind/solar energy shares): 50/30/20, 34/33/33, 20/30/50. Variant B adds the energy needed to bring imports and thermal output down to 5% of demand each.

## 6.5 Historical build pace

Generation grew 0.424 TWh/year over 2015–2025 and 0.474 TWh/year over 2020–2025. Required base-case pace to 2035 is 0.929 TWh/year, a multiple of 2.19 of the ten-year pace.

# 7 Stage 4: visualisation

Eleven figures were produced with matplotlib at 200 dpi on a light surface. The categorical colours follow a documented eight-hue default order (blue, orange, aqua, amber, magenta, green, violet, red) applied in fixed order; where a figure highlights Kenya against peers, Kenya is blue and peers are neutral grey. Text uses neutral ink, never series colours. The colour-vision validator that accompanies the palette guidance was not re-run on these specific figures; adjacent hues were used as documented, and every multi-series chart carries a legend or direct labels. Layout problems (legend overlap, label collisions, clipped legend) were found by viewing rendered images and corrected; the stress-test figure needed two passes. The fossil category in the supply-mix chart uses neutral grey and not a palette hue, to avoid implying that fossil is 'green'.

# 8 Stage 5: report generation

Reports are written with python-docx using a shared helper module (styles, headed tables, captions, callouts, code blocks) and converted to PDF with LibreOffice. Quantities in the text are f-string substitutions from the outputs, which is why the Final Report can be regenerated after a data refresh with no manual edits to numbers. After generation, pages were rendered to images and inspected; one orphaned page was fixed by merging two bullets and removing a table row.

## 8.1 Interactive dashboard (Stage 6)

src/07_build_dashboard.py reads the processed tables, packages the series as JSON and injects them into src/dashboard_template.html, producing index.html at the repository root (the GitHub Pages entry point): one self-contained file (about 80 KB, no build step, no server) that runs offline apart from the web fonts. Charts are drawn as SVG by hand-written JavaScript; there is no charting library. Four views carry live controls that recompute in the browser: forecast scenario and model comparison; pillar weights that re-rank the ten countries; a stress simulator (hydro, wind, geothermal, import and thermal-availability sliders) with a 20,000-draw seeded Monte Carlo on the observed hydrology deviations; and a capacity calculator (demand path, year, technology mix, capacity factors). The in-browser formulas are the same as in Stage 3, and the preset buttons were checked against the report: Kenya's rank under the four 70%-weight presets is 7, 3, 1, 1, the import-loss scenario returns 1.52 TWh replaced by thermal and 1.00 Mt CO2, and the phase-down option gives 2.3 GW for the base 2035 case. The browser Monte Carlo uses a different random generator from the Python one, so its shortfall probability is approximately, not exactly, the 27% in the report. The page was tested in Chromium at 1280 px and 400 px widths, in dark and light themes, with no console errors and no horizontal overflow.

# 9 Assumptions register

**Table 7. Assumptions and how each was examined**

| ID | Assumption | Affects | Check |
|---|---|---|---|
| A1 | Log-demand dynamics are well described by trend-plus-persistence over 26 years | Forecasts and intervals | Backtests; multiple model forms |
| A2 | Forecast SD grows with √h beyond h = 3 | Intervals 2029–2035 | Untestable; flagged |
| A3 | Thermal availability 65% (range 50–80%) | Stress results | Sensitivity table (Final Report 6.2) |
| A4 | Thermal capacity 0.57 GW as reported by Ember | Stress results | Not testable with data in hand |
| A5 | New plant CF: geothermal 0.85, wind 0.45, solar 0.25 | GW requirements | Mix comparison shows spread |
| A6 | Imports and thermal held at 2025 volumes in capacity gap | Gap sizes | Variant B (phase-down) |
| A7 | Equal pillar weights | Index ranks | 10,000 random weightings; corner checks |
| A8 | Historical hydrology deviations are representative of future draws (stationary climate) | Monte Carlo | Caveat; climate change would worsen tails |
| A9 | Annual energy balance is a meaningful screen | All stress results | Stated as lower bound on risk |

# 10 Decision and issue log

**Network blocked.** Downloads from GitHub raw, World Bank and Ember failed with 403 CONNECT rejections. Proxy documentation says not to route around policy denials. Resolution: the author downloaded and attached the two files; the World Bank file was made optional because OWID already carries GDP and population.

**statsmodels unavailable.** pip could not reach PyPI (DNS failure). Resolution: implemented estimators with numpy and scipy (Appendix F).

**Cluster-regression crash.** A Kenya-only regression called the clustered-SE routine with one cluster (division by zero). Resolution: Kenya-only uses classical OLS; pooled uses clustered SEs.

**Ensemble membership.** Initial four-member ensemble included the two weakest models. Changed after backtest evidence; disclosed as a selection effect (Section 5.5).

**First Monte Carlo was uninformative.** Resampling year-on-year hydro changes included growth years and produced a 0% shortfall probability, understating risk. Redesigned to resample deviations from the centred five-year mean and to condition on the import state. Results changed materially (27% shortfall probability with imports out).

**Scenario calibration.** The 'severe' drought was first set at −45% by assumption. Data showed the 2000 deviation was −42%, so S2 was recalibrated to −42% and renamed '2000-type drought'.

**Fact-check of draft text.** Draft statements were verified against the data. Corrections made: demand fell in four years (2007, 2008, 2013, 2016), not two; Sudan's per-capita demand exceeds Kenya's; concentration (HHI) is 0.5–0.9 for all neighbours except Rwanda; the claim that heavy import weighting would lower Kenya's rank was quantified (rank 7).

**Capacity data.** Zero geothermal capacity in Ember meant capacity-based reserve margins were dropped from the security index and replaced by energy-based pillars.

**Figure layout.** Stress chart legend and labels overlapped; fixed by moving the legend below the plot and adding label backgrounds. Supply-mix colours changed so fossil is grey.

**Pagination.** Chapter 2 overflowed by one bullet onto a near-empty page; tightened. Final Report is 25 pages.

# 11 Validation summary

**Table 8. Validation performed**

| Check | Result |
|---|---|
| Accounting identities (sources→generation, generation+imports→demand) | Pass, max 0.007% |
| Cross-source reconciliation | Pass, max 0.06% (not independent) |
| Out-of-sample forecasting | 42 forecasts per model; MAPE 2.9% / 3.9% / 3.3% for ensemble at h = 1 / 2 / 3 |
| Interval coverage | 80% intervals cover 87% / 93% / 100% (n = 15 / 14 / 13) |
| Index robustness | 10,000 random weightings; four corner weightings |
| Stress-test sensitivity | Thermal availability 50%, 65%, 80% |
| Text-to-data consistency | Numbers inserted programmatically; hand-written claims checked |
| Visual inspection | All figures and report pages rendered and viewed |

## 11.1 What was not validated

- No independent data source was available to confirm Ember's Kenya figures.
- The capacity-factor, thermal availability and energy-mix assumptions are not calibrated to named projects.
- The security index has no external benchmark, so it measures relative structure, not absolute risk.

# 12 Reproducing the analysis

```
# 1. clone and install
git clone <your-repo-url> && cd kenya-electricity-outlook
python3 -m pip install -r requirements.txt

# 2. run the full pipeline (under a minute; about 10 seconds in testing)
bash run_all.sh

# or stage by stage
python3 -I src/01_build_dataset.py
python3 -I src/02_forecast_models.py
python3 -I src/03_security_stress_scenarios.py
python3 -I src/04_figures.py
python3 -I src/05_build_final_report.py
python3 -I src/06_build_technical_report.py
python3 -I src/07_build_dashboard.py
```

## 12.1 Updating with new data

- Replace the two files in data/raw with newer Ember and OWID downloads (same filenames).
- Rerun run_all.sh. Check outputs/tables/data_quality.json first: identities should hold within 0.01%.
- Extend the country list in 01_build_dataset.py to apply the method to another country.

## 12.2 Extensions

- Add monthly data (generation, peak demand, reservoir levels) to move from annual energy to seasonal adequacy.
- Add cost data (tariffs, fuel prices, levelised costs) to convert the capacity gap to investment needs.
- Replace the heuristic ensemble with a formal forecast-combination study when longer or monthly series are available.
- Add a dashboard (Streamlit) that refreshes the monitoring indicators in Table 11 of the Final Report.

# Appendix E  Source code: src/01_build_dataset.py

```
"""Stage 1: build analysis-ready Kenya and regional panels from Ember + OWID raw files.
Usage: python3 -I src/01_build_dataset.py   (run from repo root)
"""
import pandas as pd, numpy as np, json

RAW = "data/raw"; OUT = "data/processed"
PEERS = {  # Ember name -> short name used in the project
    "Kenya": "Kenya", "Uganda": "Uganda", "Tanzania (the United Republic of)": "Tanzania",
    "Ethiopia": "Ethiopia", "Rwanda": "Rwanda", "Burundi": "Burundi", "Zambia": "Zambia",
    "Mozambique": "Mozambique", "Congo (DRC)": "DR Congo", "Sudan": "Sudan",
}
OWID_NAME = {"Tanzania": "Tanzania", "DR Congo": "Democratic Republic of Congo"}

e = pd.read_csv(f"{RAW}/ember_generation_yearly.csv", low_memory=False)
e = e[(e["Area type"] == "Country or economy") & e.Area.isin(PEERS)].copy()
e["country"] = e.Area.map(PEERS)
gen = e.pivot_table(index=["country", "Year"], columns="Electricity source",
                    values="Generation (TWh)", aggfunc="first")
inten = e[e["Electricity source"] == "Total generation"].set_index(["country", "Year"])[
    "Emissions intensity (gCO2e/kWh)"]
emis = e[e["Electricity source"] == "Total generation"].set_index(["country", "Year"])["Emissions (MtCO2e)"]

main = ["Hydro", "Other renewables", "Wind", "Solar", "Bioenergy", "Coal", "Gas", "Other fossil",
        "Nuclear", "Net imports", "Total generation", "Demand", "Fossil", "Clean", "Renewables"]
panel = gen[[c for c in main if c in gen.columns]].copy().fillna({"Coal":0,"Gas":0,"Nuclear":0})
panel.columns = [c.lower().replace(" ", "_") for c in panel.columns]
panel = panel.rename(columns={"other_renewables": "geothermal_other_ren", "total_generation": "generation",
                              "net_imports": "net_imports", "demand": "demand"})
panel["emissions_mtco2e"] = emis
panel["emissions_intensity_g_kwh"] = inten
panel = panel.reset_index().rename(columns={"Year": "year"})

o = pd.read_csv(f"{RAW}/owid-energy-data.csv")
o["country"] = o.country.replace({"Democratic Republic of Congo": "DR Congo"})
oc = o[o.country.isin(PEERS.values())][["country", "year", "population", "gdp",
       "electricity_demand_per_capita", "primary_energy_consumption", "energy_per_capita"]]
panel = panel.merge(oc, on=["country", "year"], how="left")
panel["demand_per_capita_kwh"] = panel.demand * 1e9 / panel.population
panel["gdp_per_capita"] = panel.gdp / panel.population
panel = panel.sort_values(["country", "year"]).reset_index(drop=True)
panel.to_csv(f"{OUT}/regional_panel.csv", index=False)

k = panel[panel.country == "Kenya"].drop(columns="country").reset_index(drop=True)
k.to_csv(f"{OUT}/kenya_annual.csv", index=False)

# --- data quality checks ---
q = {}
q["kenya_years"] = [int(k.year.min()), int(k.year.max()), int(len(k))]
q["countries"] = sorted(panel.country.unique().tolist())
# Ember generation vs OWID generation reconciliation (Kenya)
ok = o[o.country == "Kenya"].set_index("year")
kk = k.set_index("year")
rec = pd.DataFrame({"ember_gen": kk.generation, "owid_gen": ok.electricity_generation,
                    "ember_demand": kk.demand, "owid_demand": ok.electricity_demand}).dropna()
rec["gen_diff_pct"] = (rec.ember_gen / rec.owid_gen - 1) * 100
rec["dem_diff_pct"] = (rec.ember_demand / rec.owid_demand - 1) * 100
rec.to_csv("outputs/tables/data_reconciliation_kenya.csv")
q["max_abs_gen_diff_pct"] = float(rec.gen_diff_pct.abs().max())
q["max_abs_demand_diff_pct"] = float(rec.dem_diff_pct.abs().max())
# identity check: sum of sources + imports ~ demand
src = ["hydro", "geothermal_other_ren", "wind", "solar", "bioenergy", "coal", "gas", "other_fossil"]
k2 = k.copy(); k2["sum_src"] = k2[src].sum(axis=1)
q["max_abs_sources_vs_generation_pct"] = float(((k2.sum_src / k2.generation - 1) * 100).abs().max())
q["max_abs_gen_plus_imports_vs_demand_pct"] = float((((k2.generation + k2.net_imports) / k2.demand - 1) *
100).abs().max())
q["missing_by_col_kenya"] = {c: int(v) for c, v in k.isna().sum().items() if v}
q["gdp_last_year_kenya"] = int(k.year[k.gdp.notna()].max())
json.dump(q, open("outputs/tables/data_quality.json", "w"), indent=2)
print(json.dumps(q, indent=2)); print(rec.round(2).tail(6)); print(panel.shape)
```

# Appendix F  Source code: src/02_forecast_models.py

```
"""Stage 2: Kenya electricity-demand forecasting with rolling-origin backtests.
Models are implemented from scratch with numpy/scipy (no statsmodels dependency).
Usage: python3 -I src/02_forecast_models.py
"""
import numpy as np, pandas as pd, json
from scipy.optimize import minimize
from scipy import stats

k = pd.read_csv("data/processed/kenya_annual.csv").set_index("year")
panel = pd.read_csv("data/processed/regional_panel.csv")
y = np.log(k.demand)                 # model log demand (multiplicative growth)
YEARS = y.index.values

# ---------------- model implementations (each returns log-forecasts for h=1..H) ----------------
def f_drift(ly, H, **_):             # random walk with drift = mean log growth
    d = np.diff(ly).mean(); return ly[-1] + d*np.arange(1, H+1)

def f_drift_recent(ly, H, w=8, **_): # drift from last w years only
    d = np.diff(ly[-(w+1):]).mean(); return ly[-1] + d*np.arange(1, H+1)

def f_trend(ly, H, **_):             # log-linear deterministic trend (OLS)
    t = np.arange(len(ly)); b, a = np.polyfit(t, ly, 1)
    return a + b*(len(ly)-1+np.arange(1, H+1))

def f_ar1d(ly, H, **_):              # ARIMA(1,1,0) w/ drift via conditional least squares
    d = np.diff(ly); X = np.c_[np.ones(len(d)-1), d[:-1]]
    c, phi = np.linalg.lstsq(X, d[1:], rcond=None)[0]
    phi = np.clip(phi, -0.95, 0.95); out, last, cur = [], d[-1], ly[-1]
    for _ in range(H):
        last = c + phi*last; cur += last; out.append(cur)
    return np.array(out)

def _holt_sse(p, ly):
    a, b, ph = p; l, bt, sse = ly[0], ly[1]-ly[0], 0.0
    for t in range(1, len(ly)):
        yh = l + ph*bt; e = ly[t]-yh; sse += e*e
        ln = yh + a*e; bt = ph*bt + a*b*e; l = ln
    return sse
def f_holt_damped(ly, H, **_):       # ETS(A,Ad,N) on logs, params by SSE minimisation
    best = None
    for s in [(0.5,0.2,0.9),(0.8,0.1,0.8),(0.3,0.3,0.95)]:
        r = minimize(_holt_sse, s, args=(ly,), bounds=[(0.01,0.99),(0.01,0.99),(0.80,0.98)], method="L-BFGS-B")
        if best is None or r.fun < best.fun: best = r
    a, b, ph = best.x; l, bt = ly[0], ly[1]-ly[0]
    for t in range(1, len(ly)):
        yh = l + ph*bt; e = ly[t]-yh; l = yh + a*e; bt = ph*bt + a*b*e
    return np.array([l + sum(ph**i for i in range(1, h+1))*bt for h in range(1, H+1)])

MODELS = {"Drift (RW+drift)": f_drift, "Recent drift (8y)": f_drift_recent, "Log-linear trend": f_trend,
          "ARIMA(1,1,0)+drift": f_ar1d, "ETS damped trend": f_holt_damped}
# NOTE: membership chosen AFTER seeing a first backtest in which ETS-damped and recent-drift were weakest;
# this is a (documented) selection step on a small sample, so ensemble accuracy is mildly optimistic.
ENS_MEMBERS = ["Drift (RW+drift)", "ARIMA(1,1,0)+drift", "Log-linear trend"]

# ---------------- rolling-origin backtest ----------------
MIN_TRAIN, HMAX = 11, 3
rows = []
for T in range(MIN_TRAIN-1, len(y)-1):
    ly = y.values[:T+1]
    preds = {m: f(ly, HMAX) for m, f in MODELS.items()}
    preds["Ensemble (mean)"] = np.mean([preds[m] for m in ENS_MEMBERS], axis=0)
    for m, p in preds.items():
        for h in range(1, HMAX+1):
            if T+h < len(y):
                rows.append(dict(model=m, origin=int(YEARS[T]), h=h, target=int(YEARS[T+h]),
                                 actual=float(np.exp(y.values[T+h])), pred=float(np.exp(p[h-1])),
                                 logerr=float(y.values[T+h]-p[h-1])))
# driver (GDP-conditional) model: dlog D = a + b dlog GDP, GDP known ex-post (oracle) -> conditional forecast
g = np.log(k.gdp)
dd, dg = y.diff(), g.diff()
for T in range(MIN_TRAIN-1, len(y)-1):
    tr = pd.DataFrame({"dd": dd, "dg": dg}).iloc[1:T+1].dropna()
    b, a = np.polyfit(tr.dg, tr.dd, 1)
    for h in range(1, HMAX+1):
        if T+h < len(y) and not np.isnan(g.values[T+h]):
            cum = sum(a + b*(g.values[T+i]-g.values[T+i-1]) for i in range(1, h+1))
            p = y.values[T] + cum
            rows.append(dict(model="GDP-driver (conditional)", origin=int(YEARS[T]), h=h, target=int(YEARS[T+h]),
                             actual=float(np.exp(y.values[T+h])), pred=float(np.exp(p)), logerr=float(y.values[T+h]-
p)))
bt = pd.DataFrame(rows); bt["ape"] = (bt.pred/bt.actual-1).abs()*100; bt["ae"] = (bt.pred-bt.actual).abs()
bt.to_csv("outputs/tables/backtest_all_forecasts.csv", index=False)

naive_mae = bt[bt.model=="Drift (RW+drift)"].groupby("h").ae.mean()
def summarize(d):
    return pd.Series(dict(n=len(d), MAPE=d.ape.mean(), RMSE=np.sqrt(((d.pred-d.actual)**2).mean()),
                          bias_pct=((d.pred/d.actual-1)*100).mean(), sd_logerr=d.logerr.std(ddof=1)))
summ = bt.groupby(["model","h"]).apply(summarize, include_groups=False).reset_index()
summ["rel_MAE_vs_drift"] = [bt[(bt.model==r.model)&(bt.h==r.h)].ae.mean()/naive_mae[r.h] for r in summ.itertuples()]
summ.to_csv("outputs/tables/backtest_summary.csv", index=False)
```

```
# common-sample comparison (so driver model isn't judged on a different window)
common = bt[bt.target <= 2022]
cs = common.groupby(["model","h"]).apply(summarize, include_groups=False).reset_index()
cs.to_csv("outputs/tables/backtest_summary_common_window_to2022.csv", index=False)
print(summ.pivot(index="model", columns="h", values="MAPE").round(2))
print(cs.pivot(index="model", columns="h", values="MAPE").round(2))

# ---------------- panel elasticity of electricity demand to GDP (two-way growth regression) -------------
P = panel.copy(); P = P.sort_values(["country","year"])
P["dlD"] = P.groupby("country").demand.transform(lambda s: np.log(s).diff())
P["dlG"] = P.groupby("country").gdp.transform(lambda s: np.log(s).diff())
Q = P.dropna(subset=["dlD","dlG"]); Q = Q[(Q.demand>0)]
def ols_cluster(df, xcol, ycol, cl):
    X = np.c_[np.ones(len(df)), df[xcol].values]; Y = df[ycol].values
    XtXi = np.linalg.inv(X.T@X); beta = XtXi@X.T@Y; u = Y-X@beta
    meat = sum(np.outer(X[idx].T@u[idx], X[idx].T@u[idx]) for idx in [np.where(df[cl].values==c)[0] for c in
df[cl].unique()])
    G = df[cl].nunique(); n, kk = X.shape
    V = (G/(G-1))*((n-1)/(n-kk))*XtXi@meat@XtXi
    return beta, np.sqrt(np.diag(V)), G
el = {}
b, se, G = ols_cluster(Q, "dlG", "dlD", "country")   # pooled, country-clustered SEs (10 clusters: treat with
caution)
el["pooled_all_10"] = dict(intercept=float(b[0]), elasticity=float(b[1]), se=float(se[1]), n=int(len(Q)),
clusters=int(G))
# Kenya-only classical SE (n small) for reference
kd = Q[Q.country=="Kenya"]; r = stats.linregress(kd.dlG, kd.dlD)
el["kenya_only_ols"] = dict(intercept=float(r.intercept), elasticity=float(r.slope), se=float(r.stderr),
r2=float(r.rvalue**2), p=float(r.pvalue), n=int(len(kd)))
# demand per capita vs GDP per capita (levels, country FE, log-log)
L = P.dropna(subset=["demand_per_capita_kwh","gdp_per_capita"]); L = L[L.demand_per_capita_kwh>0]
L = L.assign(ly=np.log(L.demand_per_capita_kwh), lx=np.log(L.gdp_per_capita))
L["ly_d"] = L.ly - L.groupby("country").ly.transform("mean"); L["lx_d"] = L.lx -
L.groupby("country").lx.transform("mean")
b = (L.lx_d*L.ly_d).sum()/(L.lx_d**2).sum(); el["fe_loglog_levels"] = dict(elasticity=float(b), n=int(len(L)))
json.dump(el, open("outputs/tables/elasticities.json","w"), indent=2); print(json.dumps(el, indent=1))

# ---------------- final fits on full sample + interval calibration ----------------
H = 10; ly = y.values; fut = np.arange(2026, 2026+H)
fc = {m: np.exp(f(ly, H)) for m, f in MODELS.items()}
fc["Ensemble (mean)"] = np.exp(np.mean([np.log(fc[m]) for m in ENS_MEMBERS], axis=0))
ens = bt[bt.model=="Ensemble (mean)"]
sig = {h: float(np.sqrt((ens[ens.h==h].logerr**2).mean())) for h in (1,2,3)}
s1 = sig[1]; z80, z95 = stats.norm.ppf(0.9), stats.norm.ppf(0.975)
sigh = np.array([s1*np.sqrt(h) for h in range(1, H+1)])
F = pd.DataFrame({"year": fut, **{m: v for m, v in fc.items()}})
base = np.log(F["Ensemble (mean)"].values)
F["lo80"], F["hi80"] = np.exp(base-z80*sigh), np.exp(base+z80*sigh)
F["lo95"], F["hi95"] = np.exp(base-z95*sigh), np.exp(base+z95*sigh)
F.to_csv("outputs/tables/demand_forecast_statistical_2026_2035.csv", index=False)
# interval coverage check on backtest (h=1..3, ensemble): fraction inside 80/95% band
cov = {}
for h in (1,2,3):
    e_ = ens[ens.h==h]; s_ = s1*np.sqrt(h)
    cov[h] = dict(n=int(len(e_)), cover80=float((e_.logerr.abs()<=z80*s_).mean()),
cover95=float((e_.logerr.abs()<=z95*s_).mean()), empirical_rmse=sig[h], assumed_sd=float(s_))
# annualised growth recent
cagr = lambda a,b,n: (b/a)**(1/n)-1
facts = dict(cagr_2000_2025=cagr(k.demand[2000],k.demand[2025],25),
cagr_2015_2025=cagr(k.demand[2015],k.demand[2025],10),
             cagr_2020_2025=cagr(k.demand[2020],k.demand[2025],5), demand_2025=float(k.demand[2025]),
             sigma_h=sig, interval_coverage=cov, forecast_2030=float(F.set_index('year').loc[2030,'Ensemble
(mean)']),
             forecast_2035=float(F.set_index('year').loc[2035,'Ensemble (mean)']))
json.dump(facts, open("outputs/tables/forecast_facts.json","w"), indent=2)
print(json.dumps(facts, indent=1)); print(F.round(2).to_string())
```

# Appendix G Source code: src/03_security_stress_scenarios.py

```
"""Stage 3: electricity supply-security index (10-country panel), Kenya stress tests, 2035 capacity-gap scenarios.
Usage: python3 -I src/03_security_stress_scenarios.py
"""
import numpy as np, pandas as pd, json
rng = np.random.default_rng(20261012)
P = pd.read_csv("data/processed/regional_panel.csv").sort_values(["country","year"]).reset_index(drop=True)
F = pd.read_csv("outputs/tables/demand_forecast_statistical_2026_2035.csv").set_index("year")
out = {}

# ============ 1. SUPPLY-SECURITY INDEX ============
src = ["hydro","geothermal_other_ren","wind","solar","bioenergy","coal","gas","other_fossil","nuclear"]
for c in src: P[c] = P[c].fillna(0)
tot = P[src].sum(axis=1)
sh = P[src].div(tot, axis=0)
P["hhi"] = (sh**2).sum(axis=1)                                  # 1/9 (even) .. 1 (single source)
P["imp_dep"] = (P.net_imports/P.demand*100).clip(lower=0)         # % of demand met by net imports (exporters -> 0)
P["fossil_share"] = P.fossil/P.generation*100
P["hydro_share"] = P.hydro/P.generation*100
hy = P.groupby("country").hydro.transform(lambda s: s.pct_change())
P["hydro_vol"] = hy.groupby(P.country).transform(lambda s: s.rolling(8, min_periods=5).std())
P["hydro_risk"] = P.hydro_share*P.hydro_vol                       # % of generation at 1-sigma hydrology swing
comp = {"imp_dep":"Import dependence","hhi":"Source concentration (HHI)","fossil_share":"Fossil / fuel-price
exposure","hydro_risk":"Hydrology risk exposure"}
Z = P[P.year>=2006].dropna(subset=list(comp)).copy()
for c in comp:                                                    # winsorise 5/95 pooled, min-max -> 0..100 (100 =
most secure)
    lo, hi = Z[c].quantile([.05,.95]); v = Z[c].clip(lo, hi)
    Z["s_"+c] = 100*(1-(v-lo)/(hi-lo))
S = ["s_"+c for c in comp]
Z["SSI"] = Z[S].mean(axis=1)
Z["SSI_geo"] = np.exp(np.log(Z[S].clip(lower=1)).mean(axis=1))    # geometric aggregation (penalises weak pillars)
Z.to_csv("outputs/tables/security_index_panel.csv", index=False)
last = Z[Z.year==2024].set_index("country")
rank24 = last.SSI.rank(ascending=False).astype(int).sort_values()
print(last[["SSI","SSI_geo"]+S].round(1).sort_values("SSI",ascending=False))
# weight sensitivity: 10,000 Dirichlet weight vectors -> Kenya rank distribution in 2024
W = rng.dirichlet(np.ones(4), 10000); M = last[S].values
sc = M@W.T; ranks = (-sc).argsort(axis=0).argsort(axis=0)+1
kidx = list(last.index).index("Kenya"); kr = ranks[kidx]
out["ssi_2024"] = last.SSI.round(2).to_dict(); out["rank_2024"] = {k:int(v) for k,v in rank24.items()}
out["kenya_rank_dist_2024"] = {str(r): float((kr==r).mean()) for r in range(1,11) if (kr==r).any()}
out["kenya_rank_median"] = float(np.median(kr)); out["kenya_rank_p5_p95"] = [int(np.percentile(kr,5)),
int(np.percentile(kr,95))]
# Kenya trajectory + pillar change 2015 -> 2025
kz = Z[Z.country=="Kenya"].set_index("year")
out["kenya_ssi_series"] = kz.SSI.round(2).to_dict()
out["kenya_pillars_2015_2025"] = {c: [round(float(kz.loc[2015,"s_"+c]),1), round(float(kz.loc[2025,"s_"+c]),1)] for c
in comp}
out["kenya_raw_2015_2025"] = {c: [round(float(kz.loc[2015,c]),3), round(float(kz.loc[2025,c]),3)] for c in comp}
out["kenya_ssi_2025"] = float(kz.loc[2025,"SSI"]); out["kenya_ssi_2015"] = float(kz.loc[2015,"SSI"])
# rank stability across 2010-2024 (equal weights)
rk = Z[Z.year.between(2010,2024)].assign(r=lambda d:
d.groupby("year").SSI.rank(ascending=False)).query("country=='Kenya'").set_index("year").r
out["kenya_rank_by_year"] = {int(k):int(v) for k,v in rk.items()}

# ============ 2. KENYA STRESS TESTS ============
K = P[P.country=="Kenya"].set_index("year")
b = K.loc[2025]; dem = b.demand
THERM_GW, CF_THERM = 0.57, 0.65                                    # Ember fossil capacity (GW); assumed max thermal
availability
therm_ceiling = THERM_GW*8.76*CF_THERM
therm_now = b.coal+b.gas+b.other_fossil
headroom_th = therm_ceiling - therm_now
IMP_MAX = K.net_imports.max()                                      # historical max annual net imports (2025)
out["stress_inputs"] = dict(demand_2025=float(dem), thermal_2025=float(therm_now),
thermal_ceiling=float(therm_ceiling),
                            thermal_headroom=float(headroom_th), imports_2025=float(b.net_imports),
thermal_cf_assumed=CF_THERM,
                            thermal_capacity_gw=THERM_GW)
hyd_yoy = K.hydro.pct_change().dropna(); out["hydro_yoy_min"] = float(hyd_yoy.min()); out["hydro_yoy_max"] =
float(hyd_yoy.max())
out["hydro_yoy_sd"] = float(hyd_yoy.std()); out["hydro_cv_detrended"] =
float((K.hydro/K.hydro.rolling(5,center=True,min_periods=3).mean()).std())
def scenario(hydro_shock=0, imports_mult=1.0, wind_shock=0, geo_shock=0, th_cf=CF_THERM):
    ceil = THERM_GW*8.76*th_cf
    lost = -b.hydro*hydro_shock - b.wind*wind_shock - b.geothermal_other_ren*geo_shock + b.net_imports*(1-
imports_mult)
    head = max(0, ceil-therm_now); thermal_add = min(max(lost,0), head); unserved = max(0, lost-head)
    return dict(lost_twh=float(lost), thermal_add_twh=float(thermal_add), unserved_twh=float(unserved),
                unserved_pct_demand=float(unserved/dem*100), extra_co2_mt=float(thermal_add*0.658))  # 0.658
MtCO2/TWh = 658 g/kWh (Ember 'Other fossil')
sc_def = {"S0 Baseline 2025": dict(),
          "S1 Moderate drought (hydro -25%)": dict(hydro_shock=-.25),
          "S2 2000-type drought (hydro -42%)": dict(hydro_shock=-.42),
          "S3 Import interruption (Ethiopia link out)": dict(imports_mult=0.0),
          "S4 Drought -25% + imports -50%": dict(hydro_shock=-.25, imports_mult=.5),
          "S5 2000-type drought + imports out": dict(hydro_shock=-.42, imports_mult=0.0),
```

```
          "S6 Compound: S5 + wind -20% + geothermal -10%": dict(hydro_shock=-.42, imports_mult=0.0, wind_shock=-.2,
geo_shock=-.1)}
st = pd.DataFrame({k: scenario(**v) for k,v in sc_def.items()}).T;
st.to_csv("outputs/tables/stress_scenarios_2025.csv"); print(st.round(2))
# thermal CF sensitivity for S5
sens = {cf: scenario(hydro_shock=-.42, imports_mult=0.0, th_cf=cf)["unserved_pct_demand"] for cf in (0.5,0.65,0.8)}
out["stress_s5_thermal_cf_sensitivity"] = {str(k):float(v) for k,v in sens.items()}
# Monte-Carlo: resample HISTORICAL deviations of hydro and geothermal/other-renewable output from their centred 5-yr
mean
# (joint, same year => preserves co-movement) and apply them to the 2025 supply stack. Two states for imports.
dev = pd.DataFrame({"h": K.hydro/K.hydro.rolling(5,center=True,min_periods=3).mean()-1,
                    "g": K.geothermal_other_ren/K.geothermal_other_ren.rolling(5,center=True,min_periods=3).mean()-
1}).dropna()
out["hydro_dev_worst"] = {int(y):float(v) for y,v in dev.h.nsmallest(3).items()}
N = 20000; dr = dev.iloc[rng.integers(0, len(dev), N)].values
res_lost = np.clip(-(b.hydro*dr[:,0] + b.geothermal_other_ren*dr[:,1]), 0, None)
def mc(extra_lost):
    lost = res_lost + extra_lost; un = np.clip(lost-max(0, headroom_th), 0, None)
    return dict(p_any_loss=float((lost>0).mean()), p_unserved=float((un>0).mean()),
mean_unserved_twh=float(un.mean()),
                p95_unserved_pct_demand=float(np.percentile(un,95)/dem*100),
p99_unserved_pct_demand=float(np.percentile(un,99)/dem*100),
                p95_thermal_twh=float(np.percentile(np.minimum(lost, max(0,headroom_th)),95)))
out["mc_imports_available"] = mc(0.0); out["mc_imports_out"] = mc(float(b.net_imports)); out["mc_n"] = N
# ============ 3. CAPACITY-GAP SCENARIOS TO 2035 ============
CF = {"Geothermal":0.85, "Wind":0.45, "Solar":0.25}; MIXES = {"Geothermal-led (50/30/20)":
{"Geothermal":.5,"Wind":.3,"Solar":.2},
                                                             "Balanced (34/33/33)":
{"Geothermal":.34,"Wind":.33,"Solar":.33},
                                                             "Solar-heavy (20/30/50)":
{"Geothermal":.2,"Wind":.3,"Solar":.5}}
def blended_gw(twh, mix_energy): # mix given as shares of ENERGY; capacity = sum(twh*share/(8.76*CF))
    return sum(twh*s/(8.76*CF[t]) for t,s in mix_energy.items())
scn = {"Low (80% lower bound)":"lo80", "Base (ensemble)":"Ensemble (mean)", "High (80% upper bound)":"hi80"}
rows = []; base_supply = float(dem)   # 2025 supply incl. imports (1.52) and thermal (1.40) held constant
for sname, col in scn.items():
    for yr in (2030, 2035):
        d = float(F.loc[yr, col]); gap = d-base_supply
        # variant B: also phase imports down to 5% of demand and thermal down to 5% of demand -> extra requirement
        extra_B = max(0, b.net_imports-0.05*d) + max(0, therm_now-0.05*d)
        for mname, mix in MIXES.items():
            rows.append(dict(scenario=sname, year=yr, demand_twh=d, gap_twh=gap, gap_pct_2025=gap/dem*100,
                             req_new_gen_twh_per_yr=gap/(yr-2025), mix=mname,
                             new_gw_holdBAU=blended_gw(gap, mix), new_gw_phasedown=blended_gw(gap+extra_B, mix)))
cap = pd.DataFrame(rows); cap.to_csv("outputs/tables/capacity_gap_scenarios.csv", index=False)
hist_add = (K.generation[2025]-K.generation[2015])/10
out["hist_gen_addition_twh_per_yr_2015_2025"] = float(hist_add)
out["hist_gen_addition_twh_per_yr_2020_2025"] = float((K.generation[2025]-K.generation[2020])/5)
out["req_pace_base_2035"] = float((F.loc[2035,"Ensemble (mean)"]-dem)/10)
out["pace_multiple_base_2035"] = out["req_pace_base_2035"]/hist_add
print(cap[cap.mix.str.startswith("Balanced")].round(2).to_string())
# regional benchmarking table (2024)
bm = P[P.year==2024].set_index("country")
[["demand","population","demand_per_capita_kwh","hydro_share","fossil_share","imp_dep","emissions_intensity_g_kwh","h
hi"]]
g20 = P.pivot(index="year", columns="country", values="demand"); bm["cagr_2014_2024"] =
((g20.loc[2024]/g20.loc[2014])**(1/10)-1)*100
bm.round(2).to_csv("outputs/tables/regional_benchmark_2024.csv"); print(bm.round(2))
json.dump(out, open("outputs/tables/stage3_facts.json","w"), indent=2, default=float); print(json.dumps({k:out[k] for
k in out if k not in ('kenya_ssi_series',)}, indent=1, default=float))
```

# Appendix H  Source code: src/04_figures.py

```
"""Stage 4: publication figures (light surface, validated default categorical palette order)."""
import numpy as np, pandas as pd, json, matplotlib
matplotlib.use("Agg"); import matplotlib.pyplot as plt
from matplotlib.ticker import FuncFormatter
C = ["#2a78d6","#eb6834","#1baf7a","#eda100","#e87ba4","#008300","#6250d6","#e34948"]
INK, SUB, GRID, GRAY = "#0b0b0b", "#52514e", "#e6e5e1", "#a8a7a1"
plt.rcParams.update({"font.family":"DejaVu
Sans","font.size":9,"axes.edgecolor":GRID,"axes.labelcolor":SUB,"xtick.color":SUB,"ytick.color":SUB,

"axes.spines.top":False,"axes.spines.right":False,"axes.grid":True,"grid.color":GRID,"grid.linewidth":.6,"axes.axisbe
low":True,

"figure.facecolor":"#fcfcfb","axes.facecolor":"#fcfcfb","savefig.facecolor":"#fcfcfb","axes.titlesize":10.5,"axes.tit
leweight":"bold","axes.titlecolor":INK})
D="outputs/figures/"; T="outputs/tables/"
k = pd.read_csv("data/processed/kenya_annual.csv").set_index("year"); P =
pd.read_csv("data/processed/regional_panel.csv")
F = pd.read_csv(T+"demand_forecast_statistical_2026_2035.csv").set_index("year")
Z = pd.read_csv(T+"security_index_panel.csv"); f3 = json.load(open(T+"stage3_facts.json"))
def save(fig, n): fig.tight_layout(); fig.savefig(D+n, dpi=200); plt.close(fig)

# F1 generation mix (stacked) + demand line
fig, ax = plt.subplots(figsize=(7.2,4))
k["fossil_all"] = k.coal.fillna(0)+k.gas.fillna(0)+k.other_fossil.fillna(0)
st = [("Geothermal & other ren.","geothermal_other_ren"),("Hydro","hydro"),("Wind","wind"),("Solar","solar"),
("Bioenergy","bioenergy"),("Fossil (thermal)","fossil_all"),("Net imports","net_imports")]
ax.stackplot(k.index, *[k[c].clip(lower=0) for _,c in st], colors=[C[0],C[1],C[2],C[3],C[4],"#6b6a64",C[6]],
labels=[n for n,_ in st], edgecolor="#fcfcfb", linewidth=.8)
ax.plot(k.index, k.demand, color=INK, lw=1.4, ls="--", label="Electricity demand")
ax.set_ylabel("TWh per year"); ax.set_title("Kenya's electricity supply mix, 2000–2025"); ax.set_xlim(2000,2025)
ax.legend(ncol=2, frameon=False, fontsize=8, loc="upper left"); save(fig,"fig01_supply_mix.png")

# F2 forecast fan
fig, ax = plt.subplots(figsize=(7.2,4))
ax.plot(k.index, k.demand, color=C[0], lw=2, label="Observed demand")
ax.fill_between(F.index, F.lo95, F.hi95, color=C[0], alpha=.12, lw=0, label="95% interval"); ax.fill_between(F.index,
F.lo80, F.hi80, color=C[0], alpha=.25, lw=0, label="80% interval")
ax.plot(F.index, F["Ensemble (mean)"], color=C[0], lw=2, ls="--", label="Ensemble forecast"); ax.axvline(2025.5,
color=GRAY, lw=.8)
for yr in (2030,2035): ax.annotate(f'{F.loc[yr,"Ensemble (mean)"]:.1f} TWh', (yr,F.loc[yr,"Ensemble (mean)"]),
xytext=(-8,12), textcoords="offset points", color=INK, fontsize=8.5, ha="right")
ax.set_ylabel("TWh per year"); ax.set_title("Kenya electricity demand: history and 2026–2035 forecast");
ax.legend(frameon=False, loc="upper left"); save(fig,"fig02_forecast_fan.png")

# F3 backtest accuracy
s = pd.read_csv(T+"backtest_summary.csv"); order = s[s.h==1].sort_values("MAPE").model.tolist()
fig, ax = plt.subplots(figsize=(7.2,3.8)); w=.27; x=np.arange(len(order))
for i,h in enumerate((1,2,3)):
    v=[s[(s.model==m)&(s.h==h)].MAPE.iloc[0] for m in order]; ax.barh(x+(1-i)*w, v, height=w-.03, color=C[i],
label=f"{h}-year ahead")
ax.set_yticks(x); ax.set_yticklabels(order); ax.invert_yaxis(); ax.set_xlabel("Mean absolute % error (rolling-origin
backtest)"); ax.set_title("Out-of-sample accuracy by model and horizon"); ax.legend(frameon=False);
save(fig,"fig03_backtest.png")

# F4 per-capita demand vs GDP per capita (2022)
d = P[(P.year==2022)].dropna(subset=["gdp_per_capita","demand_per_capita_kwh"])
fig, ax = plt.subplots(figsize=(6.4,4.2))
for r in d.itertuples():
    kn = r.country=="Kenya"; ax.scatter(r.gdp_per_capita, r.demand_per_capita_kwh, s=70 if kn else 40, color=C[0] if
kn else GRAY, zorder=3, edgecolor="#fcfcfb")
    ax.annotate(r.country, (r.gdp_per_capita, r.demand_per_capita_kwh), xytext=(5,4), textcoords="offset points",
fontsize=8, color=INK if kn else SUB)
ax.set_xscale("log"); ax.set_yscale("log"); ax.set_xlabel("GDP per capita (2011 int-$, log scale)");
ax.set_ylabel("Electricity demand per capita (kWh, log scale)")
ax.set_title("Electricity demand per capita vs income, 2022"); ax.xaxis.set_major_formatter(FuncFormatter(lambda v,_:
f"{v:,.0f}")); ax.yaxis.set_major_formatter(FuncFormatter(lambda v,_: f"{v:,.0f}"));
save(fig,"fig04_percapita_vs_gdp.png")

# F5 security index 2024 + Kenya trajectory
z24 = Z[Z.year==2024].sort_values("SSI"); fig, (a,b) = plt.subplots(1,2,figsize=(8.4,3.8),
gridspec_kw={"width_ratios":[1,1.1]})
a.barh(z24.country, z24.SSI, color=[C[0] if c=="Kenya" else GRAY for c in z24.country], height=.65)
for i,v in enumerate(z24.SSI): a.text(v+1, i, f"{v:.0f}", va="center", fontsize=8, color=INK)
a.set_xlim(0,100); a.set_title("Supply-security index, 2024"); a.set_xlabel("0 = least secure, 100 = most secure")
for c in Z.country.unique():
    z = Z[Z.country==c]; b.plot(z.year, z.SSI, color=C[0] if c=="Kenya" else GRAY, lw=2.4 if c=="Kenya" else 1,
zorder=3 if c=="Kenya" else 1)
b.text(2025.3, Z[(Z.country=="Kenya")&(Z.year==2025)].SSI.iloc[0], "Kenya", color=C[0], fontsize=9, va="center",
fontweight="bold")
b.set_title("Index trajectory 2006–2025 (grey = peers)"); b.set_ylabel("Index"); b.set_xlim(2006,2027);
save(fig,"fig05_security_index.png")

# F6 Kenya pillars 2015 vs 2025
pil = f3["kenya_pillars_2015_2025"]; nm = {"imp_dep":"Import dependence","hhi":"Source
concentration","fossil_share":"Fossil / fuel-price exposure","hydro_risk":"Hydrology risk exposure"}
fig, ax = plt.subplots(figsize=(6.8,3.2))
for i,(c,(a0,a1)) in enumerate(pil.items()):
    ax.plot([a0,a1],[i,i], color=GRAY, lw=2, zorder=1); ax.scatter(a0,i,color=GRAY,s=60,zorder=3,label="2015" if i==0
else None); ax.scatter(a1,i,color=C[0],s=60,zorder=3,label="2025" if i==0 else None)
    ax.text(max(a0,a1)+2, i, f"{a0:.0f} → {a1:.0f}", va="center", fontsize=8.5, color=INK)
```

```
ax.set_yticks(range(4)); ax.set_yticklabels([nm[c] for c in pil]); ax.invert_yaxis(); ax.set_xlim(40,115);
ax.set_xlabel("Pillar score (100 = most secure)"); ax.set_title("Kenya: what moved in the security index, 2015 →
2025"); ax.legend(frameon=False, loc="lower left"); save(fig,"fig06_pillars.png")

# F7 import dependence + hydro/geothermal shares (Kenya)
fig, (a,b) = plt.subplots(1,2,figsize=(8.4,3.6))
a.bar(k.index[-14:], (k.net_imports/k.demand*100)[-14:], color=C[1], width=.7); a.set_title("Net imports as % of
demand"); a.set_ylabel("%")
sh = lambda c: k[c]/k.generation*100
for nme,c,i in [("Geothermal & other","geothermal_other_ren",0),("Hydro","hydro",1),("Wind","wind",2)]:
    b.plot(k.index, sh(c), color=C[i], lw=2, label=nme)
b.set_title("Share of generation (%)"); b.legend(frameon=False, fontsize=8); save(fig,"fig07_imports_shares.png")

# F8 stress scenarios
st = pd.read_csv(T+"stress_scenarios_2025.csv", index_col=0).iloc[1:]
fig, ax = plt.subplots(figsize=(7.4,3.9)); y=np.arange(len(st))
ax.barh(y, st.thermal_add_twh, color=C[3], height=.62, label="Covered by extra thermal generation")
ax.barh(y, st.unserved_twh, left=st.thermal_add_twh, color=C[7], height=.62, label="Unserved energy",
edgecolor="#fcfcfb", linewidth=1.5)
ax.axvline(f3["stress_inputs"]["thermal_headroom"], color=INK, lw=1, ls="--", label="Thermal headroom limit (1.85
TWh)")
for i,r in enumerate(st.itertuples()): ax.text(r.lost_twh+.05, i, f"{r.unserved_pct_demand:.1f}% unserved" if
r.unserved_pct_demand>0 else "fully covered", va="center", fontsize=8, color=INK, bbox=dict(facecolor="#fcfcfb",
edgecolor="none", pad=1.2), zorder=5)
ax.set_yticks(y); ax.set_yticklabels([s_.split(" (")[0] if len(s_)>40 else s_ for s_ in st.index], fontsize=8);
ax.invert_yaxis(); ax.set_xlim(0,5); ax.set_xlabel("Lost supply to be replaced (TWh per year)")
ax.set_title("Stress tests on the 2025 supply stack"); ax.legend(frameon=False, loc="upper center", fontsize=8,
bbox_to_anchor=(0.2,-0.2), ncol=2); save(fig,"fig08_stress.png")

# F9 Monte Carlo summary
a_, b_ = f3["mc_imports_available"], f3["mc_imports_out"]
fig, (a,b) = plt.subplots(1,2,figsize=(7.6,3.3))
a.bar(["Imports\navailable","Imports\ninterrupted"], [a_["p_unserved"]*100, b_["p_unserved"]*100], color=[C[0],C[1]],
width=.55); a.set_title("Probability of an annual energy shortfall"); a.set_ylabel("%")
for i,v in enumerate([a_["p_unserved"]*100,b_["p_unserved"]*100]): a.text(i, v+.8, f"{v:.0f}%", ha="center",
fontsize=9, color=INK)
b.bar(["Imports\navailable","Imports\ninterrupted"], [a_["p95_unserved_pct_demand"], b_["p95_unserved_pct_demand"]],
color=[C[0],C[1]], width=.55); b.set_title("95th-percentile shortfall (% of demand)")
for i,v in enumerate([a_["p95_unserved_pct_demand"], b_["p95_unserved_pct_demand"]]): b.text(i, v+.15, f"{v:.1f}%",
ha="center", fontsize=9, color=INK)
save(fig,"fig09_monte_carlo.png")

# F10 capacity gap
cp = pd.read_csv(T+"capacity_gap_scenarios.csv"); cp = cp[cp.year==2035]; fig, ax = plt.subplots(figsize=(7.2,3.8));
sc=["Low (80% lower bound)","Base (ensemble)","High (80% upper bound)"]; mx=cp.mix.unique(); w=.26
for i,m in enumerate(mx):
    v=[cp[(cp.scenario==s_)&(cp.mix==m)].new_gw_holdBAU.iloc[0] for s_ in sc]; ax.bar(np.arange(3)+(i-1)*w, v,
width=w-.03, color=C[i], label=m)
    for j,vv in enumerate(v): ax.text(j+(i-1)*w, vv+.06, f"{vv:.1f}", ha="center", fontsize=8, color=INK)
ax.set_xticks(range(3)); ax.set_xticklabels(["Low demand","Base demand","High demand"]); ax.set_ylabel("New capacity
needed by 2035 (GW)"); ax.set_title("New generation capacity needed by 2035 under three build mixes");
ax.legend(frameon=False, fontsize=8, title="Energy mix (geo/wind/solar)", title_fontsize=8);
save(fig,"fig10_capacity_gap.png")

# F11 rank distribution under random weights
rd = f3["kenya_rank_dist_2024"]; fig, ax = plt.subplots(figsize=(5.6,3.2)); ax.bar([int(r) for r in rd], [v*100 for v
in rd.values()], color=C[0], width=.65)
ax.set_xlabel("Kenya's rank among 10 countries (1 = most secure)"); ax.set_ylabel("% of 10,000 random weightings");
ax.set_title("Is Kenya's rank an artefact of the weights?"); ax.set_xticks(range(1,10));
save(fig,"fig11_rank_sensitivity.png")
print("figures done")
```